Co-Scheduled Adjoint Greeks in a Fixed-Point FPGA Datapath: Heston-Model Option Sensitivities with First-Order Rounding Bounds

Yash Agrawal
Birla Institute of Technology and Science, Pilani
Pilani, India

Govind Prasad
Birla Institute of Technology and Science, Pilani
Pilani, India

***Abstract*—FPGA option-pricing engines obtain Greeks, the sensitivities of a price to its inputs, by bump-and-reprice or by re-simulation. We present an FPGA engine for European options under the Heston model, priced with the COS Fourier-cosine method, in which a hand-derived reverse (adjoint) sweep is co-scheduled with the forward pass in one statically scheduled fixed-point datapath. Because the COS loop has no data-dependent control flow, the values the reverse sweep needs are held in a statically allocated, register-resident tape rather than in memory. A generator emits, from one operation graph, a bit-accurate model, the modulo-scheduled Verilog and a first-order rounding-error bound for the price and each of nine first-order Greeks, used at design time to size and certify the word length. Implemented in Vivado on a Zynq-7020 at equal throughput, the price-and-Greeks engine takes 1.03× the cycles of a price-only engine at 1.68× its LUTs and 2× its DSP blocks; a factored forward-mode engine built by the same flow needs 7% more LUTs, 12% more flip-flops and 25% more DSP blocks and is 6× less accurate at the same word length. The bound was never exceeded over 21,000 inputs, and the engine ran bit-exact on two FPGA families (1,469/1,469 outputs, including 50 calibrations to NIFTY options). The bound covers rounding only; the COS truncation error dominates in a few in-domain cases. The computed engine latency on a ZCU102, 15.9 μs excluding bus transfers, is 4.9–15× below general-purpose AD and bump-and-reprice on a laptop core but not below a vectorised implementation of hand-derived Greeks (13.4 μs).**

***Index Terms*—FPGA, adjoint differentiation, option Greeks, Heston model, COS method, fixed-point arithmetic, rounding-error analysis, modulo scheduling**

I. Introduction

Option Greeks, the partial derivatives of a price with respect to spot, strike, maturity, rate and the model's parameters, set hedge ratios and risk limits and drive model calibration. Under the Heston stochastic-volatility model [1] the price has no closed form and is computed numerically, here by the COS method [2].

On FPGAs, published engines compute Greeks by bump-and-reprice or by re-running the simulation for each sensitivity [14], [15], [21]. Nine Greeks by central differences cost 19 pricings, and the result depends on a bump size with no setting that is safe for every Greek (Fig. 7). Software replaced bumping with adjoint algorithmic differentiation (AAD) [5]–[7], which returns the derivatives of one output with respect to all inputs in a single reverse sweep, at the cost of recording a tape of the forward computation; our taped CPU baseline costs 7.6 pricings.

The reverse sweep of a computation with no data-dependent control flow can be scheduled statically, a property exploited by source-transformation AD [10] and by neural-network training accelerators. The COS pricer has this property: every one of its 128 terms runs the same operations. We use it to co-schedule a hand-derived reverse sweep with the forward pass in one fixed-point datapath, and ask what the Greeks then cost in time, area and accuracy, measured against the alternatives built through the same flow. The contributions are:

1) **A co-scheduled adjoint datapath** for Heston–COS Greeks, in which forward values live in a statically allocated, register-resident tape (Section III). After a search of the FPGA and finance-acceleration literature (Section II) we know of no prior FPGA computation of option Greeks by an adjoint, and no prior FPGA implementation of the COS method.
2) **A measured cost of the Greeks**: three engines of equal throughput (price only, adjoint, factored forward mode) implemented in Vivado through the same generator and flow, with resources, cycles and fixed-point accuracy side by side (Section VI-A).
3) **A first-order rounding-error bound for every Greek** of a datapath that contains its own reverse sweep, extending AD-based precision analysis for FPGAs [25], [26] to a datapath that itself differentiates; with the scope of the bound, the modelling of the arithmetic primitives and its empirical validation stated explicitly (Sections III-C, VI-E).
4) **A generator and bit-exact silicon results** on two FPGA families, including calibrations to real market data (Section VI-G).

What is not claimed: the reverse sweep is derived by hand, not by an AD tool; the bound covers rounding only and is

computed at design time, not by the engine; and the engine is not faster than a vectorised CPU implementation of hand-derived Greeks (Section VII).

II. Background and Related Work

A. Heston model, the COS method and its Greeks

Under the risk-neutral measure the Heston model is $\mathrm{d}S_t = rS_t\,\mathrm{d}t + \sqrt{v_t}S_t\,\mathrm{d}W_1$, $\mathrm{d}v_t = \kappa(\theta - v_t)\,\mathrm{d}t + \xi\sqrt{v_t}\,\mathrm{d}W_2$, $\mathrm{d}W_1\mathrm{d}W_2 = \rho\,\mathrm{d}t$, with nine pricing inputs $p = (S_0, K, T, r, v_0, \kappa, \theta, \xi, \rho)$. We use the characteristic function φ of the log-price in the form of Albrecher et al. [4], which avoids the complex-logarithm discontinuity. The COS method [2] expands the density on a truncation range $[a, b]$ in a cosine series; for a European put

$$V_{\mathrm{put}} \approx Ke^{-rT} \sum_{k=0}^{N-1}{}' \,\mathrm{Re}\Big[\varphi(u_k)\, e^{\,iu_k(x-a)}\Big]\, U_k, \quad u_k = \tfrac{k\pi}{b-a}, \tag{1}$$

with $x = \ln(S_0/K)$, closed-form payoff coefficients U_k, and $\sum'$ halving the first term. Calls follow from put–call parity. The N terms are independent.

Delta and Gamma have closed-form COS expressions from the same sum [2, Remark 3.2], analysed by Junike [3]: since x enters (1) only through $e^{iu_k x}$, $\partial/\partial S_0$ multiplies each term by iu_k/S_0, and $\partial/\partial K$ follows from x and the linearity of U_k in K. These two Greeks are therefore nearly free in any COS implementation. The derivatives with respect to T, r and the five model parameters $(v_0, \kappa, \theta, \xi, \rho)$ enter through φ, and these are where differentiation does real work.

B. Computing Greeks

Bump-and-reprice approximates $\partial V/\partial p_j$ by central differences: 19 pricings for nine Greeks, with truncation error $O(h^2)$ and rounding error $O(\epsilon/h)$. Forward-mode AD carries one tangent per input direction. Reverse-mode (adjoint) AD propagates $\bar{v} = \partial V/\partial v$ backwards from the output and returns all derivatives in one sweep at a small constant multiple of the pricing cost [8]; software tools store the forward values on a tape, or recompute them under checkpointing [9]. For the Heston–Fourier price, Cui et al. [12] derived the gradient by hand.

C. Hardware acceleration of option pricing

FPGA work on option pricing covers Monte Carlo for the Heston model [17], [18], binomial and trinomial trees [19], and multi-dimensional quadrature [20]. For Greeks, Klaisoongnoen et al. ported the STAC-A2 risk benchmark (Heston Monte Carlo with Greeks by re-simulation, i.e. finite differences) to Alveo, Stratix-10 and Versal devices [14]–[16], and AMD's Vitis Quantitative Finance Library computes Greeks by central differences [21]. The COS method has been accelerated on GPUs, for multiple strikes and the Heston model [22], [23], and adjoint AD runs on GPUs with a tape [24]. Table I positions this work; the reported figures measure different things and are not directly comparable. We found no FPGA computation of Greeks by an adjoint and no FPGA COS implementation after searching FPL, FCCM, FPT, ReConFig, TRETS, the HEART and H2RC workshops and general indexes; absence from a search is not proof of absence.

D. Precision analysis for FPGA datapaths

Sizing fixed-point word lengths from sensitivities obtained by automatic differentiation was introduced for FPGAs by Gaffar et al. [25], and accuracy-guaranteed bit-width optimisation by affine arithmetic by Lee et al. [26]; Gappa [27] gives machine-checked error enclosures. Our bound is Linnainmaa's first-order analysis [11], applied to a datapath that contains its own adjoint operations, so that it bounds the rounding error of each Greek as the hardware computes it. We do not compare its tightness with an affine-arithmetic or Gappa enclosure.

III. Method

A. A register-resident tape

A reverse sweep needs the forward values in reverse order; software AD stores them on a tape. In (1) the control flow does not depend on the input: every term runs the same operations, the truncation range is fixed per evaluation, and there are no data-dependent branches. The forward and reverse computations of one term therefore form a fixed operation graph whose every value has a lifetime known at design time, and the modulo scheduler allocates for each value the rotating registers that hold it across the terms in flight. The tape is thus compiled into the schedule: it costs registers, not memory traffic. Over all terms in flight, the 56-bit adjoint datapath needs 447 such registers, against 150 for a price-only datapath of the same throughput and 374 for the factored forward datapath; in Vivado the Greeks add 11,901 flip-flops and 1,129 shift-register LUTs to the price-only engine (Section VI-A).

The Greeks returned are the derivatives of the COS approximation, including the put–call parity terms, with $[a, b]$ held fixed (A4 in Table IV). Per term the reverse sweep propagates adjoints from the term's contribution back to the model inputs, reusing the forward pass's reciprocals; a power-of-two normalisation keeps the small adjoints of high-frequency terms at full precision; nine accumulators sum the price and the Greek contributions.

B. Arithmetic

The datapath has no dividers. Reciprocals and square roots use Newton–Raphson iterations from table seeds, the exponential Tang's method [28], the logarithm a table and a short series, and sines, cosines and complex magnitudes and angles CORDIC [29]. Each COS term costs 250 multiplications, three CORDIC rotations and two CORDIC vectorings (126 multiplications for the price alone). Values are two's-complement fixed point with FL fraction bits (56-bit words with $FL = 28$, or 64 with $FL = 32$), rounded to nearest after every operation. Data-dependent shifts are built only as wide as the ranges measured over the input domain with margin, which cut logic by 35%; an input that would exceed a range raises a `range_err` flag rather than return a wrong value.

TABLE I
POSITIONING AGAINST RELATED WORK. REPORTED RESULTS ARE EACH PAPER'S OWN HEADLINE FIGURES AND ARE NOT DIRECTLY COMPARABLE.

Work	Platform	Model / pricing	Greeks computed by	Accuracy guarantee	Reported result
de Schryver et al. [17], [18]	FPGA (Virtex-6)	Heston, (multi-level) Monte Carlo	— (price only)	none	energy-efficient pricing
Jin et al. [19]	FPGA (Virtex-4)	binomial / trinomial trees	—	none	>160× vs. one CPU core
Tse et al. [20]	FPGA, GPU	multi-dimensional quadrature	—	none	quadrature-core design space
Klaisoongnoen et al. [14]–[16]	Alveo U280, Stratix-10, Versal	STAC-A2: Heston MC, Longstaff–Schwartz	re-simulation (finite differences)	none (precision explored)	8–185× efficiency vs. 2× Xeon 8260M [14]
AMD Vitis Quant. Library [21]	Alveo (HLS)	Heston MC; Heston closed form	central differences	none	library
Zhang & Oosterlee [22], [23]	GPU	COS, incl. Heston, many strikes	— (price only)	none	GPU speed-up of COS
Gremse et al. [24]	GPU	general	taped adjoint AD	none	AAD on GPUs
Cui et al. [12]	CPU	Heston, Fourier	analytic gradient (by hand)	—	faster than numerical gradient
This work	Zynq-7020, Zynq US+	Heston, COS (N=128), 56/64-bit fixed point	**hand-derived adjoint, co-scheduled**, 9 Greeks in one pass	**first-order rounding bound per Greek**	1.03× cycles, 1.68× LUT of price-only; bit-exact on silicon

C. First-order rounding-error bound

Let δ_i be the local error introduced at operation i, with $|\delta_i| \le \epsilon_i$. To first order the error in an output y is $\sum_i (\partial y/\partial w_i)\,\delta_i$, where w_i is the result of operation i, so

$$|\hat{y} - y| \;\le\; \sum_i \left|\frac{\partial y}{\partial w_i}\right| \epsilon_i \;=:\; B_y, \tag{2}$$

Linnainmaa's first-order analysis [11]. The sensitivities are obtained by a reverse sweep over the operation graph, which includes the hardware's own adjoint operations, so B_y bounds the rounding error of each Greek as the hardware computes it. Products of local errors are dropped; they are about 2^{-FL} times smaller (Section VI-E).

What the bound covers. B_y bounds the difference between the fixed-point datapath and the *same graph* evaluated in exact arithmetic. The primitives are expanded into elementary operations in the graph (the Newton–Raphson iterations, table lookups and series terms), so every internal rounding has its own $\epsilon_i = 2^{-(FL+1)}$; their approximation error (finite iterations, table resolution, series truncation) is common to the fixed-point and exact evaluations of the graph and is measured separately: at most 1.1×10^{-8} relative, against an exact-function reference, on the 21-case grid of Section VI-E. Each CORDIC unit is one operation whose local error was measured against its exact integer model over 30,000 inputs: rotation outputs are within 1.14 ulp, modelled as $\epsilon = 2$ ulp; vectoring magnitudes within 0.8 ulp; the vectoring angle loses precision as the input magnitude falls (at most $1.75/|z|$ ulp), and is modelled as $\epsilon = 2\max(1, 1/|z|)$ ulp at the evaluation point. The bound excludes the COS truncation error (Section VI-F).

How it is used. B_y is computed in software from the graph in 0.1 s per parameter set, so the engine does not attach it to each result. It serves at design time to choose the word length and to certify, over a sampled input domain, the accuracy the engine delivers (Table VIII).

D. Generator and modulo scheduling

Fig. 1 shows the flow. One Python description of setup (once per evaluation), term ($N = 128$ times) and finish (once) is an operation graph from which the generator derives the bit-accurate model, a double-precision model, the bound (2) and a modulo schedule [30] emitted as Verilog. The testbench compares every register at the cycle it becomes valid against the model (999 to 2,004 checks per design), so every property shown for the model holds for the hardware. The terms are independent, so a new term starts every initiation interval (II) while earlier ones are in flight (Fig. 2). With 8 multipliers, 3 CORDIC rotators and 2 vectorers, all three resource classes give $\mathrm{II} = \lceil 250/8 \rceil = 3{\cdot}32/3 = 2{\cdot}32/2 = 32$ cycles, and the loop takes $127{\cdot}32{+}506{+}2 = 4{,}572$ cycles. The CORDIC units set the pace, so the reverse sweep's 124 extra multiplications per term fit into multiplier slots the forward pass leaves idle — but only because the price-and-Greeks engine has twice the multipliers a price-only engine needs for the same II: the price alone needs $\lceil 126/32 \rceil = 4$.

IV. IMPLEMENTATION

Three configurations come from the same generator (Table II). In the host-setup configuration the setup and finish steps (about 100 multiplications and a few transcendental functions per evaluation) run on the host as generated C that is bit-exact with the model, and the FPGA runs the 128-term loop; the 64-bit configuration computes everything on chip. Each engine sits behind an AXI4-Lite register file, driven for testing by a JTAG-to-AXI master with no processor software. Table III gives the board implementations.

V. EXPERIMENTAL METHODOLOGY

Input sets. (i) A 21-case grid over moneyness, maturity, vol-of-vol and correlation, calls and puts. (ii) A 10,500-input sweep: 6,000 uniform over the verified domain (S_0=100, $K \in$

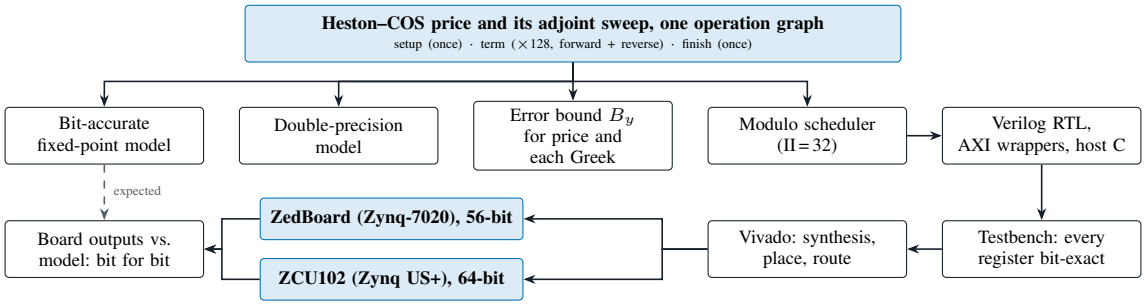


Fig. 1. Design flow. One operation graph yields the bit-accurate model, the error bound and the scheduled RTL; the RTL is checked register by register against the model, implemented in Vivado and run on two boards, whose outputs are compared bit for bit with the same model.

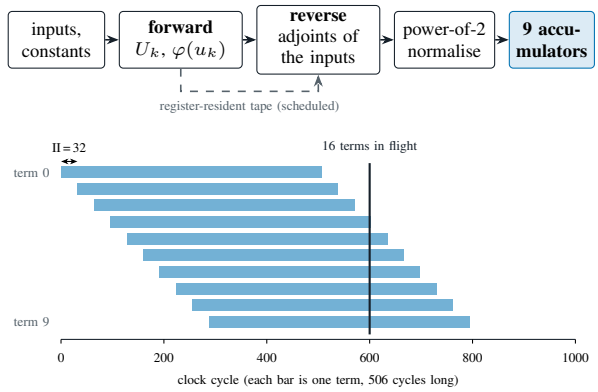


Fig. 2. Top: one COS term; the reverse sweep shares the forward pass's datapath and reads forward values from scheduled rotating registers. Bottom: the modulo schedule (first 10 of 128 terms); 8 multipliers and 5 CORDIC units are shared by about 16 overlapping terms.

TABLE II
ENGINE CONFIGURATIONS (CYCLES FROM CYCLE-EXACT RTL SIMULATION)

Config.	Word / frac.	Mult.	On chip	Cycles
`z7`	56 / 28	8	all	4,733
`z7h`	56 / 28	8	128-term loop	4,572
`zu`	64 / 32	32	all	1,593

TABLE III
BOARD IMPLEMENTATIONS (VIVADO, POST-ROUTE, JTAG TEST SYSTEMS)

	ZedBoard	ZCU102	ZCU104
Device	xc7z020-1	xczu9eg-2	xczu7ev-2
Engine	`z7h`	`zu`	`zu`
LUT	38,752 (72.8%)	77,515 (28.3%)	77,863 (33.8%)
FF	32,134 (30.2%)	63,547 (11.6%)	63,796 (13.8%)
DSP	72 (32.7%)	510 (20.2%)	510 (29.5%)
BRAM tiles	4.5 (3.2%)	7.5 (0.8%)	6.5 (2.1%)
Clock (MHz)	70	100	100
WNS (ns)	+0.458	+1.601	+1.469
$F_{\max}$ (MHz)	≈79[a]	≈119	≈117
Run on silicon	yes	yes	no

[a] Engine alone, out of context; the 56-bit carry chains limit the clock. Totals include the JTAG-to-AXI master and interconnect. Vivado 2025.2 (ZedBoard, ZCU104) and 2026.1 (ZCU102).

$[60, 150]$, $T \in [0.1, 3]$, $r \in [0, 0.1]$, $v_0, \theta \in [0.005, 0.25]$, $\kappa \in [0.2, 6]$, $\xi \in [0.1, 1]$, $\rho \in [-0.95, 0.6]$), 500 in each of seven hard regimes (Feller condition violated, ρ near its limits, deep strikes, very short and long maturities, high vol-of-vol, very low variance) and 1,000 with one input outside the domain. (iii) A held-out sweep of the same composition with a fresh seed, drawn after the shift ranges were fitted. (iv) 50 board cases (2 fixed, 48 random). (v) 50 calibrations of the model to NIFTY index options on 50 trading days.

References. Rounding error is measured against the same graph in double precision; the COS result is compared separately with an independent Fourier-integral price (method error).

Equal-throughput engines. Three host-setup engines with II = 32 were generated, checked bit-exact against their models by the same testbench generator, and implemented out of context in Vivado 2025.2 on the xc7z020clg484-1 at 70 MHz: price only (4 multipliers), adjoint (8) and factored forward mode (10). The forward datapath carries its tangents without the per-term rescaling the adjoint applies to its adjoints, as a forward-mode AD tool would; this leaves it less accurate, not larger.

CPU baselines. The identical algorithm in C++ (double precision, g++ 13) on an Intel Core i5-1155G7 laptop (4 cores, 8 threads): price only, bump-and-reprice (19 pricings), an optimised bump that reuses the characteristic function, forward-mode AD (9 directions in one pass), taped adjoint AD with CoDiPack [13], and hand-derived analytic Greeks after Cui et al. [12], all with `-O3 -march=native`. In that build the compiler leaves the 128-term loop scalar. Two vectorised variants of the analytic method, built with `-Ofast -fopenmp-simd` and glibc's vector math library, compute across terms: the single loop made branch-free, which vectorises but spills its live complex temporaries and runs slower than the scalar code, and the same work split into seven short loops over arrays of the 128 terms (structure of arrays), which vectorises cleanly and is the vectorised baseline reported. Every method matches the reference model (adjoint to 1.3×10^{-10} relative; the vectorised variants agree with forward mode to 9.3×10^{-12}). Latency is the best of five passes over 2,000 inputs on one core of an idle machine; throughput uses eight independent processes.

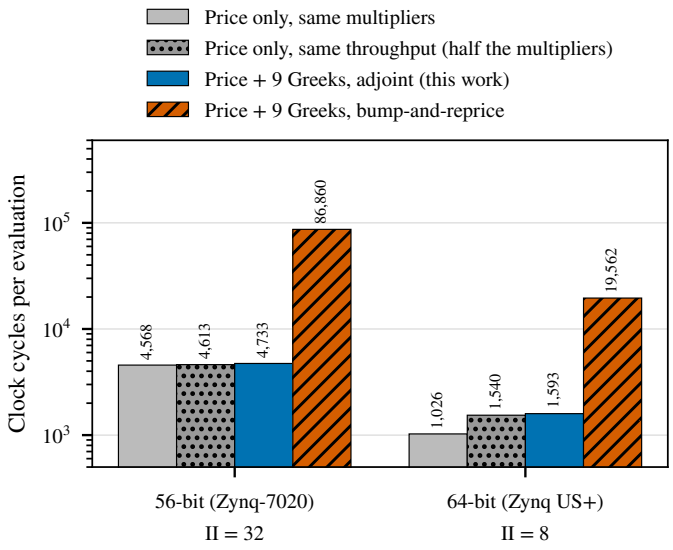


Fig. 3. Clock cycles per evaluation (cycle-exact RTL simulation, 128 COS terms): price only on the same hardware as the adjoint engine and at the same initiation interval with half the multipliers, the adjoint engine, and bump-and-reprice.

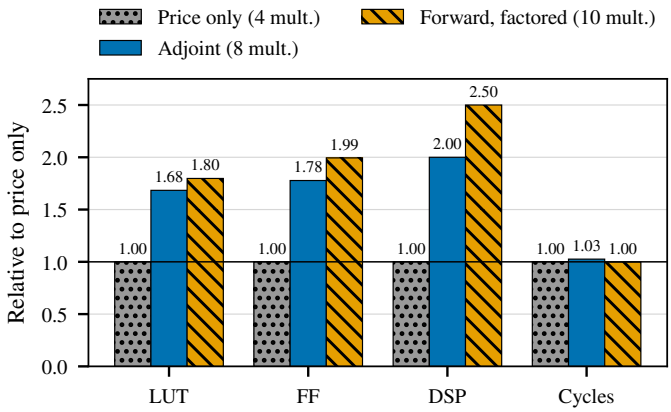


Fig. 4. The three engines of Table V, normalised to the price-only engine.

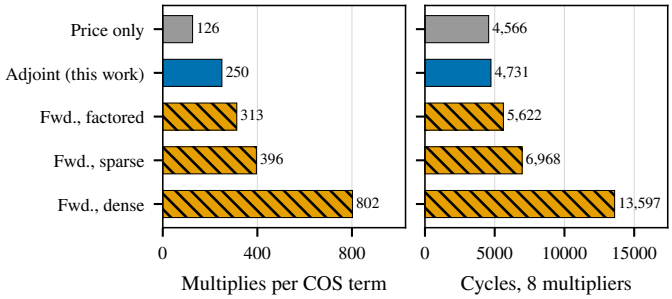


Fig. 5. Price and nine Greeks by datapath type, 56-bit design: multiplies per COS term and cycles with 8 shared multipliers (scheduler).

A. Assumptions and their rationale

Table IV lists every modelling and measurement assumption, why it was made, and how its effect was checked.

VI. Results

A. Cost of the Greeks

Fig. 3 gives cycles per evaluation. On the same hardware the price and nine Greeks take 1.04× the price alone on the 56-bit design (4,733 vs. 4,568 cycles) and 1.55× on the 64-bit design, where the 32 multipliers rather than the CORDIC units set the pace. These figures flatter the adjoint: a price-only engine reaches the same II with half the multipliers. At equal throughput the Greeks cost about 3% in cycles on both configurations (4,733 vs. 4,613; 1,593 vs. 1,540) and twice the multipliers.

Table V and Fig. 4 give the area at equal throughput, from Vivado. Relative to a right-sized price-only engine, the price and nine Greeks cost 1.03× the cycles, 1.68× the LUTs, 1.78× the flip-flops and 2× the DSP blocks; the Zynq-7020 design is LUT-bound. For comparison, hand-derived analytic Greeks cost 1.5 pricings in time on the CPU. Of the 11,901 flip-flops the Greeks add, the register-resident tape accounts for most: 297 more rotating 56-bit registers (about 16.6 kbit) than the price-only schedule, part of which Vivado maps to shift-register LUTs (1,129 more).

B. Adjoint against forward mode

Three forward-mode (tangent) datapaths were built by the same generator, with the same primitives, range and scheduler: *dense* (all eight directions through every operation, as a vector forward-mode AD tool does; a weak baseline, shown for completeness), *sparse* (structural zeros skipped) and *factored* (derivatives of the exponent with respect to its inner complex quantities formed once per term, as a hand derivation of analytic Greeks does). The factored one is the competitor that matters. All return the same price and Greeks as the adjoint to 6×10^{-14} in double precision. Per term the adjoint needs 250 multiplications, the factored forward datapath 313 (Fig. 5, Table VI). With 8 multipliers this is 4,731 against 5,622 cycles; with 16, all but the dense datapath are CORDIC-bound, and the factored one is slightly faster (4,571 against 4,619). At equal throughput (Table V) the factored forward engine is 2.7% faster in cycles but needs 6.8% more LUTs, 12.2% more flip-flops and 25% more DSP blocks, and at the same word length it is 6× less accurate (worst Greek error 7.4×10^{-5} against 1.2×10^{-5}), because its tangents are not rescaled per term. The adjoint's advantage over a well-factored forward datapath is therefore real but modest: about a fifth of the multipliers, a few percent of the logic, and accuracy.

C. Design space

Fig. 6 sweeps the number of shared 64-bit multipliers. Cycles halve as multipliers double until the dependency chain of a term dominates beyond about 64; at equal multipliers the adjoint takes 1.1–1.9× the cycles of the price and 9.8–18× fewer than bump-and-reprice.

TABLE IV
ASSUMPTIONS, THEIR RATIONALE, AND HOW EACH WAS CHECKED

	Assumption	Why it was made	How its effect was checked or bounded
A1	Bounded input domain (Section V)	Fixed-point shifters are sized from ranges measured over a domain; an unbounded domain would need floating point.	Inputs outside raise `range_err`; held-out sweep: no in-domain flag, every unflagged out-of-domain result within its bound.
A2	S_0 normalised to 100	The price is homogeneous of degree one in (S_0, K); the host rescales inputs and outputs.	Exact identity; the NIFTY cases were rescaled this way.
A3	COS with N=128 and range $c_1 \pm 10\sqrt{c_2}$	Standard choices for Heston–COS [2]; 128 fixes the pipeline length.	Method error measured against a Fourier integral; it dominates in a few cases (Section VI-F).
A4	Range $[a, b]$ held fixed when differentiating	$[a, b]$ is a numerical parameter, not a model input; the CPU reference uses the same convention.	Greeks compared with an independent Fourier-integral reference; differences within the method error.
A5	Puts priced; calls by put–call parity	Call payoff coefficients grow like e^b and cancel in fixed point.	Worst call price error 75× smaller with parity.
A6	First-order bound (products of local errors dropped)	Second-order terms are $\approx 2^{-FL}$ smaller.	50-digit evaluation: error $\leq$0.24 of the bound; bound still holds at 44 bits, where second-order terms are 4,096× larger.
A7	Local errors: $2^{-(FL+1)}$ per rounding; CORDIC as in Section III-C	Round-to-nearest is what the RTL implements; the CORDIC models were measured.	RTL bit-exact with the model; CORDIC errors measured over 30,000 inputs per unit.
A8	Word lengths 56/64 bits	On the 21-case grid, 56 bits is the shortest word with every Greek's worst relative error $\leq 1.2 \times 10^{-5}$ (52 bits: 5.8×10^{-4}); a 56×56 multiplier maps to 9 DSP48 blocks.	The grid understates the tail: on the held-out sweep $\partial V/\partial\xi$ keeps 3.0 significant figures at the 99th percentile and 1.4 at worst (Table VII).
A9	Latency = cycle count × clock period, engine only	Over JTAG each register access takes milliseconds, so the board run checks outputs but cannot time the engine.	Cycle counts are exact; the bus transfers (19 words in, 20 out) and, for `z7h`, the host's setup and finish are excluded and stated.
A10	FPGA power from simulated switching activity of the routed engine	Vivado's default 12.5% toggle rate underestimated dynamic power 3–6×.	Routed netlist simulated gate by gate on four bit-exact evaluations; chip-level boundary stated in Section VI-H.
A11	CPU package power from **[HWiNFO measurement]**	The CPU energy needs a measured power.	**[measured CPU package power, idle and under load]**
A12	First-order Greeks only; vega is $\partial V/\partial v_0$	The adjoint gives first derivatives; Gamma is second order. $\partial V/\partial\sigma_0 = 2\sigma_0\,\partial V/\partial v_0$.	Gamma has a closed-form COS expression [2] that would add one accumulator; not built.
A13	Adjoint written by hand in the operation graph	Automatic derivation in the generator is future work.	Agrees with three forward-mode datapaths to 6×10^{-14} and with CoDiPack to 1.3×10^{-10}.
A14	NIFTY cases: calibrated parameters, $r = 0.06$	Realistic inputs for the silicon test, not a trading evaluation.	All 50 inside the domain; 500/500 outputs bit-exact.

TABLE V
THREE ENGINES OF EQUAL THROUGHPUT (II = 32), ZYNQ-7020, OUT OF CONTEXT, 70 MHZ

Engine	Mult.	LUT	FF	DSP	BRAM	Cycles	Worst err.
Price only	4	21,251	15,303	36	0	4,457	—
Adjoint	**8**	**35,782**	**27,204**	**72**	6	4,572	$\mathbf{1.2 \times 10^{-5}}$
Fwd., factored	10	38,199	30,514	90	2	4,449	7.4×10^{-5}

All three generated, checked bit-exact against their models, and implemented in the same flow (WNS +0.377, +0.109, +0.275 ns). Shift-register LUTs: 284, 1,413, 911. Worst error: worst relative error of any Greek over the 21-case grid against the same graph in double precision.

TABLE VI
CYCLES PER EVALUATION BY DATAPATH AND SHARED MULTIPLIERS (SCHEDULER)

		56-bit		64-bit	
Datapath	**Mult./term**	**8**	**16**	**16**	**32**
Price only	126	4,566	4,561	1,538	1,024
Adjoint (this work)	**250**	**4,731**	4,619	**2,612**	**1,591**
Forward, factored	313	5,622	4,571	3,069	1,795
Forward, sparse	396	6,968	4,571	3,690	2,171
Forward, dense	802	13,597	7,150	7,155	3,822

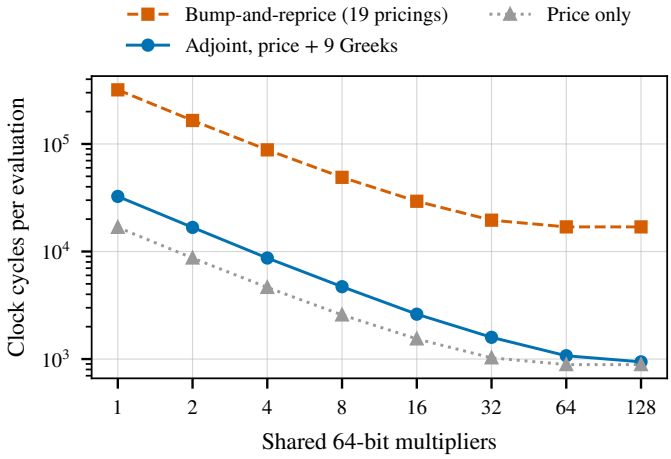


Fig. 6. Cycles against the number of shared multipliers (64-bit configuration, scheduler).

D. Against bump-and-reprice

On the same hardware, bump-and-reprice (19 pricings) needs 18.4× the adjoint's cycles on the 56-bit design and

TABLE VII
SIGNIFICANT FIGURES DELIVERED, 56-BIT DESIGN, 9,500 HELD-OUT IN-DOMAIN INPUTS

Out.	Med.	P99	Worst	Out.	Med.	P99	Worst
V	7.6	5.6	4.4	∂_{v_0}	7.6	5.3	3.3
∂_{S_0}	8.1	5.7	3.9	∂_κ	6.0	3.5	2.2
∂_K	7.9	5.7	3.9	∂_θ	7.2	4.9	3.5
∂_T	7.6	5.5	4.0	∂_ξ	5.9	3.0	1.4
∂_r	7.9	5.9	4.4	∂_ρ	7.0	4.9	3.7

$-\log_{10}$ of the relative error against the same graph in double precision; inputs where a Greek is within 1% of zero are excluded.

TABLE VIII
THE BOUND RELATIVE TO EACH VALUE, 56-BIT DESIGN, 9,500 HELD-OUT IN-DOMAIN INPUTS

Out.	Med.	P99	Max	Out.	Med.	P99	Max
V	1e-6	6e-5	6e-4	∂_{v_0}	8e-7	2e-4	4e-2
∂_{S_0}	3e-7	5e-5	3e-3	∂_κ	3e-5	4e-3	7e-2
∂_K	7e-7	6e-5	3e-3	∂_θ	2e-6	3e-4	6e-3
∂_T	1e-6	9e-5	3e-3	∂_ξ	4e-5	2e-2	1.1
∂_r	6e-7	4e-5	7e-4	∂_ρ	4e-6	3e-4	6e-3

$B_y/|y|$, the bound (2) divided by the value; same exclusion as Table VII. A sample of the domain, not a supremum over it.

12.3× on the 64-bit design. A fairer bump baseline takes Delta and $\partial V/\partial K$ from the COS sum in closed form (Section II) and bumps only the other seven inputs: 15 pricings, about 14.5× the adjoint's cycles on the 56-bit design (estimated from the measured cost per pricing; not built). Fig. 7 shows the accuracy: bump-and-reprice error is U-shaped in the bump size, with the optimum at a different size for each Greek, and the adjoint is 32–560× more accurate (median 145×) than bump-and-reprice even with the best bump size chosen afterwards for each Greek.

E. Rounding accuracy and the bound

The bound covers rounding only; Section VI-F shows that the COS method's own error can be far larger. Fig. 8 gives, on the 21-case grid, the worst relative rounding error of each output against the same graph in double precision, and the worst error as a fraction of its bound. On the 56-bit design the price is accurate to 6.1×10^{-7} and the Greeks to at most 1.2×10^{-5}; the 64-bit design gains one to two orders of magnitude; no output reaches more than 0.103 of its bound.

The grid understates the tail. Over the 10,500-input sweep no unflagged result exceeded its bound (worst 0.378) and there was no overflow; on the held-out sweep (Fig. 9) no in-domain input raised `range_err`, the worst in-domain ratio was 0.356, and every unflagged out-of-domain result stayed within its bound. Two checks support the first-order assumption (A6): evaluated in 50-digit arithmetic on 100 held-out inputs, the hardware error is at most 0.24 of the bound; and the bound still holds at 44 bits, where second-order terms are 4,096× larger. Table VII gives the significant figures delivered on the held-out inputs, and Table VIII the bound itself, relative to each value, as an a-priori accuracy guide over the sampled domain. The price and the market Greeks keep at least 5.3 figures for 99% of inputs; $\partial V/\partial\kappa$ and $\partial V/\partial\xi$, small and computed through long chains with cancellation, keep 3.5 and 3.0 at the 99th percentile and only 2.2 and 1.4 at worst, and for $\partial V/\partial\xi$ the bound can exceed the value.

F. COS method error

On the 50 board cases the COS result was compared with an independent Fourier-integral price. The method error has a median of 2×10^{-11} for the price, but in a few in-domain cases it far exceeds the bounded rounding error: for vega it reaches 7.8×10^{-3} at $\xi=1$, $T=0.34$, where 128 terms are too few (256 remove it), and a deep out-of-the-money put needs a 1.5× wider range. The engine does not flag these cases. The total error against the Heston model is at most the rounding bound plus the method error, and only the former is bounded; any implementation of this COS configuration shares the method error. A deployment would restrict the domain where 128 terms suffice or use $N=256$, which roughly doubles the cycles.

TABLE IX
SILICON VALIDATION (OUTPUTS COMPARED BIT FOR BIT WITH THE MODEL)

Board	Input set	Cases	Bit-exact	Err./bound
ZedBoard	vector + sweep	1 + 50	9 + 450 / 459	0.199
ZCU102	vector	1	10 / 10	0.099
ZCU102	sweep	50	500 / 500	0.243
ZCU102	real NIFTY calibrations	50	500 / 500	0.255
Total			**1,469 / 1,469**	≤ 0.255

ZedBoard: 9 raw sums per case, finished on the host by bit-exact code. ZCU102: 10 outputs per case computed on chip. No `range_err`.

G. Silicon validation

Table IX summarises the board runs. Cases ran back to back with no reset between them; every output of every case matched the bit-accurate model, so the accuracy results above, including the bound, hold on silicon. The worst error/bound ratio on silicon was 0.255, and the worst price error against double precision 7.3×10^{-7} on the 64-bit engine.

H. Against a CPU: latency, throughput and energy

Fig. 10 and Table X compare the same algorithm on the FPGA and on one laptop core. The FPGA figures are computed engine latencies (A9): they exclude the bus transfers (39 register accesses) and, for the ZedBoard, the host's setup and finish, while the CPU figures are measured whole evaluations. Against the general-purpose methods the ZCU102 engine is faster: 4.9× than forward-mode AD, 6.5× than taped adjoint AD and 15× than bump-and-reprice. Against hand-derived analytic Greeks it is 1.3× faster than the scalar build (20.3 μs) but slower than the vectorised one: at 13.4 μs, one vectorised core is 1.19× faster than the engine's computed latency, before the engine's omitted transfers are counted; the ZedBoard's loop is 3.2× slower than the scalar build. We therefore claim no latency advantage over a well-optimised CPU implementation. For throughput the scalar build is the CPU's best: with eight

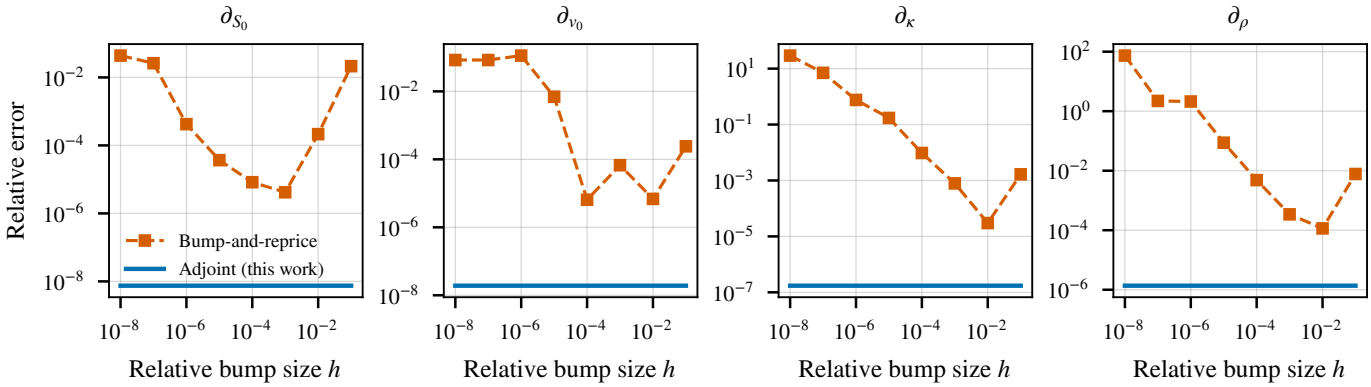


Fig. 7. Relative error of bump-and-reprice (central differences) against relative bump size, and of the adjoint engine, for four Greeks (56-bit design, base case). The other five Greeks behave alike.

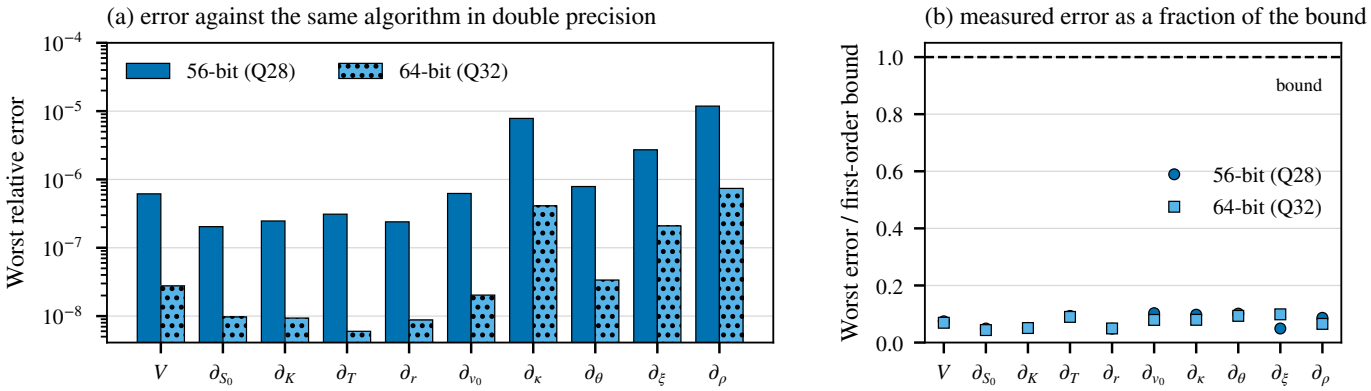


Fig. 8. Rounding accuracy over the 21-case grid. (a) Worst relative error of the price V and each Greek against the same graph in double precision. (b) Worst measured error divided by the first-order bound (2); 1 would mean the bound is met with equality.

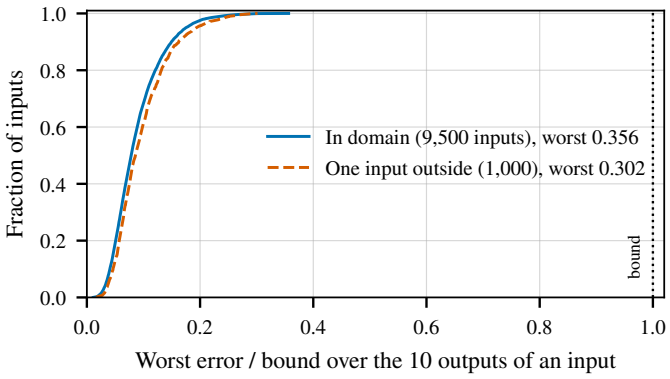


Fig. 9. Held-out 10,500-input sweep: distribution of the worst error/bound ratio over the ten outputs of each input (unflagged inputs; 56-bit design).

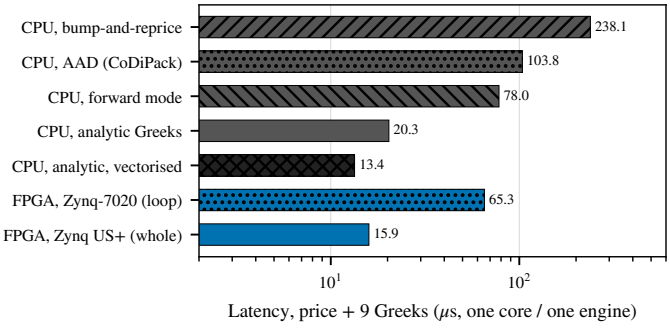


Fig. 10. Latency of one price and nine Greeks: one CPU core (Intel i5-1155G7, idle machine; measured) and one FPGA engine (cycle count × clock period, bus transfers excluded; A9).

processes on the four cores the vectorised build reaches only 77,000 evaluations per second, against 136,000 for the scalar build in the same session (158,000 in the main session of Table X). One 64-bit engine gives 62,800; by LUTs, three engines would fit on the ZCU102 (a projection, not built), giving about 188,000 per second.

Energy. Both sides are counted at chip level: for the FPGA, the device's on-chip power with the engine as its only logic (engine dynamic power from simulated activity, plus the whole device's static power): 0.676 W on the Zynq-7020 and 2.485 W for the 64-bit engine, giving 44 and 40 μJ per evaluation; for the CPU, the package power. Neither includes DRAM or the board, and the FPGA side excludes the host processor and the bus that a deployed system needs, so the FPGA figure is optimistic. The CPU, at its measured package power of

TABLE X
LATENCY, THROUGHPUT AND ENERGY PER EVALUATION (PRICE + 9 GREEKS)

Method	μs	Cost	Eval./s	Energy
CPU, price only	13.7	1×	—	—
CPU, bump-and-reprice	238.1	17.4×	—	—
CPU, optimised bump	146.6	10.7×	—	—
CPU, adjoint (CoDiPack)	103.8	7.6×	26,900[a]	—
CPU, forward mode	78.0	5.7×	34,000[a]	—
CPU, analytic Greeks	20.3	1.5×	158,000[a]	**[HWiNFO]**[b]
CPU, analytic, vectorised[e]	13.4	—	77,000[a]	**[HWiNFO]**[b]
FPGA, ZedBoard, 70 MHz (loop)[d]	65.3	1.04×	15,300	44 μJ[c]
FPGA, ZCU102, 100 MHz[d]	15.9	1.55×	62,800	40 μJ[c]

Cost: time relative to the price alone on the same platform and hardware. [a]4 cores, 8 processes. [b]Measured package power under the 8-process load divided by throughput: **[P_{pkg} W / throughput = E μJ]**; single core **[P_1 W × latency]**. [c]Chip-level, from simulated activity (A10): 0.676 W × 65.3 μs and 2.485 W × 15.9 μs. [d]Computed engine latency; bus transfers excluded. [e]Structure-of-arrays variant, measured in a separate session in which the scalar analytic method took 21.8 μs and gave 136,000 s^{-1}.

[HWiNFO: P_{pkg} W] under full load, uses **[E_{CPU} μJ]** per evaluation at its full throughput, so the FPGA engine uses **[ratio]**× less energy per evaluation on this accounting; a board-level measurement would be needed to claim a system-level energy advantage.

VII. DISCUSSION AND LIMITATIONS

What the results support. For a pricer without data-dependent control flow, a hand-derived reverse sweep can share one statically scheduled datapath with the forward pass, with its tape in scheduled registers. At equal throughput this costs about 1.7× the area of a price-only engine at 1.03× its cycles, and it is cheaper in multipliers and logic, and more accurate at the same word length, than a well-factored forward-mode datapath built by the same flow. Each Greek has a first-order rounding bound that held on every input tested and on silicon. These are the claims; speed against a CPU is not.

What they do not. The adjoint's advantage over a factored forward datapath is modest (Section VI-B). Delta and the strike sensitivity are nearly free in any COS implementation, so the adjoint's work is in the seven Greeks that enter through φ. The bound does not cover the COS method error, which dominates in a few in-domain cases, and it is a design-time tool, not attached to each result. Against a vectorised CPU implementation of hand-derived Greeks the engine is not faster, and its latency was computed, not measured. Gamma, the most-used second-order Greek, is not computed (A12).

Strike chains. All strikes of one expiry share the COS grid, so one characteristic function and one reverse sweep per term can serve every strike, each extra strike costing about 7% of one option; this was scheduled and emulated (32 strikes in 4,892 cycles on the 64-bit design) but not built, and the same sharing helps a CPU (2.3 μs per strike at 32 strikes on one core).

Further limitations. The Zynq-7020 design closes timing at 79 MHz, not 100 MHz; the input domain is bounded (A1); and end-to-end latency over the processor bus and board-level power have not been measured.

VIII. CONCLUSION

For the COS method, whose operations do not depend on the input, a hand-derived reverse sweep can be co-scheduled with the forward pass in one fixed-point datapath, with its tape held in statically allocated registers. Measured against price-only and forward-mode engines of equal throughput built by the same flow, the nine Greeks cost 1.03× the cycles and about 1.7× the area of the price, with fewer resources and better accuracy than the forward-mode alternative; each Greek carries a first-order rounding bound that was never exceeded, and the engine ran bit-exact on two FPGA families, including on real market calibrations. Deriving the reverse sweep automatically, adding Gamma, measuring end-to-end latency and board power, and placing several engines per device are the next steps.

REFERENCES

[1] S. L. Heston, "A closed-form solution for options with stochastic volatility with applications to bond and currency options," *Rev. Financial Stud.*, vol. 6, no. 2, pp. 327–343, 1993.

[2] F. Fang and C. W. Oosterlee, "A novel pricing method for European options based on Fourier-cosine series expansions," *SIAM J. Sci. Comput.*, vol. 31, no. 2, pp. 826–848, 2008.

[3] G. Junike, "On the number of terms in the COS method for European option pricing," *Numer. Math.*, 2024, doi:10.1007/s00211-024-01402-1.

[4] H. Albrecher, P. Mayer, W. Schoutens, and J. Tistaert, "The little Heston trap," *Wilmott Mag.*, pp. 83–92, Jan. 2007.

[5] M. Giles and P. Glasserman, "Smoking adjoints: Fast Monte Carlo Greeks," *Risk*, vol. 19, pp. 88–92, Jan. 2006.

[6] L. Capriotti, "Fast Greeks by algorithmic differentiation," *J. Comput. Finance*, vol. 14, no. 3, pp. 3–35, 2011.

[7] L. Capriotti and M. Giles, "15 years of adjoint algorithmic differentiation (AAD) in finance," *Quant. Finance*, vol. 24, no. 9, pp. 1353–1379, 2024.

[8] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA, USA: SIAM, 2008.

[9] A. Griewank, "Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation," *Optim. Methods Softw.*, vol. 1, no. 1, pp. 35–54, 1992.

[10] L. Hascoët and V. Pascual, "The Tapenade automatic differentiation tool: Principles, model, and specification," *ACM Trans. Math. Softw.*, vol. 39, no. 3, 2013.

[11] S. Linnainmaa, "Taylor expansion of the accumulated rounding error," *BIT Numer. Math.*, vol. 16, no. 2, pp. 146–160, 1976.

[12] Y. Cui, S. del Baño Rollin, and G. Germano, "Full and fast calibration of the Heston stochastic volatility model," *Eur. J. Oper. Res.*, vol. 263, no. 2, pp. 625–638, 2017.

[13] M. Sagebaum, T. Albring, and N. R. Gauger, "High-performance derivative computations using CoDiPack," *ACM Trans. Math. Softw.*, vol. 45, no. 4, 2019.

[14] M. Klaisoongnoen, N. Brown, and O. Thomson Brown, "Low-power option Greeks: Efficiency-driven market risk analysis using FPGAs," in *Proc. Int. Symp. Highly-Efficient Accelerators and Reconfigurable Technologies (HEART)*, 2022, pp. 95–101.

[15] M. Klaisoongnoen, N. Brown, and O. Thomson Brown, "Fast and energy-efficient derivatives risk analysis: Streaming option Greeks on Xilinx and Intel FPGAs," in *Proc. Int. Workshop Heterogeneous High-Performance Reconfigurable Computing (H2RC), SC22*, 2022.

[16] M. Klaisoongnoen, N. Brown, T. Dykes, J. R. Jones, and U.-U. Haus, "Evaluating Versal AI Engines for option price discovery in market risk analysis," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2024.

[17] C. de Schryver *et al.*, "An energy efficient FPGA accelerator for Monte Carlo option pricing with the Heston model," in *Proc. Int. Conf. Reconfigurable Computing and FPGAs (ReConFig)*, 2011.

[18] C. de Schryver, P. Torruella, and N. Wehn, "A multi-level Monte Carlo FPGA accelerator for option pricing in the Heston model," in *Proc. Design, Automation & Test in Europe (DATE)*, 2013, pp. 248–253.

[19] Q. Jin, D. B. Thomas, W. Luk, and B. Cope, "Exploring reconfigurable architectures for tree-based option pricing models," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 2, no. 4, art. 21, 2009.

[20] A. H. T. Tse, D. Thomas, and W. Luk, "Design exploration of quadrature methods in option pricing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 20, no. 5, pp. 818–826, 2012.

[21] AMD, "Vitis Libraries: Quantitative Finance Library," [Online]. Available: https://github.com/Xilinx/Vitis_Libraries

[22] B. Zhang and C. W. Oosterlee, "Option pricing with COS method on graphics processing units," in *Proc. IEEE Int. Symp. Parallel & Distributed Processing (IPDPS)*, 2009, pp. 1–8.

[23] B. Zhang and C. W. Oosterlee, "Acceleration of option pricing technique on graphics processing units," *Concurrency Computat.: Pract. Exper.*, vol. 26, no. 9, pp. 1626–1639, 2014.

[24] F. Gremse, A. Höfter, L. Razik, F. Kiessling, and U. Naumann, "GPU-accelerated adjoint algorithmic differentiation," *Comput. Phys. Commun.*, vol. 200, pp. 300–311, 2016.

[25] A. A. Gaffar, O. Mencer, W. Luk, P. Y. K. Cheung, and N. Shirazi, "Floating-point bitwidth analysis via automatic differentiation," in *Proc. IEEE Int. Conf. Field-Programmable Technology (FPT)*, 2002, pp. 158–165.

[26] D.-U. Lee, A. A. Gaffar, R. C. C. Cheung, O. Mencer, W. Luk, and G. A. Constantinides, "Accuracy-guaranteed bit-width optimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 10, pp. 1990–2000, 2006.

[27] F. de Dinechin, C. Lauter, and G. Melquiond, "Certifying the floating-point implementation of an elementary function using Gappa," *IEEE Trans. Comput.*, vol. 60, no. 2, pp. 242–253, 2011.

[28] P. T. P. Tang, "Table-driven implementation of the exponential function in IEEE floating-point arithmetic," *ACM Trans. Math. Softw.*, vol. 15, no. 2, pp. 144–157, 1989.

[29] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, 1959.

[30] B. R. Rau, "Iterative modulo scheduling: An algorithm for software pipelining loops," in *Proc. 27th Annu. Int. Symp. Microarchitecture (MICRO)*, 1994, pp. 63–74.